

2-3 · MongoDB 구축 및 비정형 로그 적재 · 이론 1H

NoSQL과 문서 지향 데이터베이스 개념

표에 안 맞는 데이터는 어디에 담는가 — 문서·컬렉션

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1시간 뒤에는

- ① 정형 데이터와 비정형 데이터의 차이를 접속 로그로 설명한다
- ② 문서=행, 컬렉션=테이블, _id=기본키로 대응해 말할 수 있다
- ③ 스키마가 없다는 것의 장점과 위험을 함께 말한다
- ④ 「이 데이터는 MySQL 과 MongoDB 중 어디에?」를 판단한다

무엇이 더 좋은가가 아니라 데이터 모양에 따라 답이 갑니다 — 다음 1H 실습에서 직접 넣어 봅니다

칸이 정해진 표 vs 자유 양식 메모 묶음

MySQL 테이블 — 표

열을 미리 정하고 모든 행이 같은 칸을 갖는다

- 새 필드가 생기면 ALTER TABLE 로 열 추가
- 아니면 모든 경우의 열을 미리 만들고 대부분 NULL
- 종류가 계속 늘어나는 로그에는 둘 다 부담

VS

MongoDB 컬렉션 — 메모 묶음

메모마다 적는 항목이 달라도 한 묶음에 보관한다

- 이벤트 종류마다 필드가 달라도 그대로 넣는다
- 값 안에 객체·배열을 그대로 담는다
- 접속 로그·이벤트 기록에 편하다

2. 기본 낱말

RDB 와 나란히 놓고 보기

MongoDB	MySQL 로 치면	설명
문서(Document)	행(row)	JSON 과 거의 같은 BSON, 저장 최소 단위
컬렉션(Collection)	테이블	문서를 모아 두는 곳, 스키마를 미리 정하지 않음
데이터베이스	데이터베이스	컬렉션을 담는 상위 단위
필드(Field)	컬럼	문서마다 있을 수도, 없을 수도 있음
_id	PRIMARY KEY	자동으로 붙는 고유 식별자(ObjectId)
인덱스	인덱스	개념·목적이 같음
aggregate 파이프라인	SELECT + GROUP BY	단계를 이어 붙여 집계

값 안에 또 객체나 배열을 넣을 수 있다 — 표에서는 어려운 일이고, 문서 DB 의 큰 장점

3. 스키마가 없다는 것

자유이자 위험 — 검증 책임이 앱으로 옮겨온다

장점

데이터 모양이 바뀌어도 DDL 없이 바로 넣는다

- 로그·이벤트처럼 형태가 변하는 데이터에 강하다
- 중첩 구조를 그대로 담는다 — 표라면 쪼개고 JOIN 할 것을 한 문서에

단점

오타·타입 실수를 DB 가 잡아 주지 않는다

VS

- 적재하는 코드 쪽에서 검증해야 한다
- 여러 컬렉션을 넘나드는 복잡한 조인은 RDB 보다 불편
- 보완: JSON Schema validator 를 중요한 컬렉션에 건다

3. 스키마가 없다는 것

level 을 Level 로 잘못 쓰면 — 오류 없이 결과만 빈다

1

잘못 쓴 문서가 그냥 들어간다

2-1 에서 「DB 가 잘못된 데이터를 막아 준다」고 했지만 MongoDB 는 기본적으로 막지 않는다

2

find({ level: ... }) 가 그 문서를 못 찾는다

Level 과 level 은 다른 필드 — 오류도 나지 않고 결과만 비어 나온다

3

그래서 알아채기 어렵다

실습 6단계에서 일부러 틀려 넣어 이 실체를 눈으로 확인한다

구조를 지키는 책임

DB 가 아니라

앱 코드 쪽

4. 판단 기준

이런 데이터라면 어디에 담을까

이런 데이터라면	어디에	왜
주문·회원·결제처럼 관계가 얽힌 것	MySQL	조인이 잦고 정합성이 돈과 직결
형태가 고정되고 집계를 자주 하는 것	MySQL	SQL 분석 도구 생태계가 넓다
접속 로그·이벤트·센서 기록	MongoDB	종류마다 필드가 다르고 계속 쌓인다
외부 API 응답 원본 보관	MongoDB	JSON 그대로 넣고 나중에 파싱
상품 상세처럼 항목이 제각각인 것	MongoDB	카테고리마다 속성이 다르다
둘 다 필요한 경우	둘 다	원본은 문서로, 집계 결과는 표로

이번 과정도 로그 원본은 MongoDB, 정제 결과는 4주차 Iceberg 표, 분석은 5주차 Spark — 도구마다 잘하는 자리가 다르다

5. aggregate 파이프라인

단계를 이어 붙인다 — 1-1 의 파이프(l)와 같은 발상



\$match 를 맨 앞에 두는 것이 성능의 기본 — 먼저 걸러야 뒤 단계가 처리할 문서가 줄고 인덱스도 쓴다

단계 일곱 개 — SQL 로 치면

MongoDB 단계	SQL 로 치면	하는 일
\$match	WHERE	조건에 맞는 문서만 남긴다
\$group	GROUP BY + 집계함수	묶어서 센다
\$sort	ORDER BY	정렬
\$limit / \$skip	LIMIT / OFFSET	개수 제한
\$project	SELECT 컬럼 목록	보여 줄 필드를 고르고 가공
\$lookup	LEFT JOIN	다른 컬렉션을 붙인다
\$unwind	(배열 펼치기)	배열 하나를 여러 문서로 편다

실습 1.5H 에서 \$match → \$group → \$sort 와 \$dateToString·\$cond 를 직접 돌린다

이 1시간에서 꼭 가져갈 것

- 1 문서=행, 컬렉션=테이블, _id=기본키로 대응해 이해한다
- 2 스키마가 없다는 것은 자유이자 위험 — 검증 책임이 앱으로 옮겨온다
- 3 중첩 구조를 그대로 담을 수 있는 것이 문서 DB 의 큰 장점이다
- 4 무엇이 더 좋은가가 아니라 데이터 모양에 따라 고른다
- 5 aggregate 는 \$match → \$group → \$sort 가 WHERE → GROUP BY → ORDER BY

2-3 · 이론 1H 마무리

다음은 MongoDB 기동과 mongosh 실습 (1H)

컨테이너를 띄우고 mongosh 로 문서를 넣고 찾고 고치고 지우는 한 바퀴를 돕니다

✓ docker run 으로 MongoDB 7.0 기동

✓ mongosh 접속 · use logdb

✓ insertMany·find·updateOne·deleteOne

✓ 필드 이름을 일부러 틀려 넣어 보기

준비할 것

Docker Desktop 실행·터미널 열기